🗺️ UFC Fighter Pattern Analysis & Fight Predictor: Comprehensive Roadmap

Phase 1: Database Architecture & Environment Setup

- **Goal:** Establish a relational database schema designed for temporal pattern tracking and ML feature extraction without data leakage.
- **Duration:** 1 – 2 Days

Tasks & Action Items

- [ ] **Initialize Local Storage:** Set up a local database instance using PostgreSQL or SQLite.
- [ ] **Build Schema:** Execute standard DDL scripts creating tables with strict foreign key constraints:
    - fighters: Base physical traits (fighter_id, name, height_cm, reach_cm, dob, stance).
    - events: Event dates and locations (event_id, event_name, event_date).
    - fights: Individual matchups (fight_id, event_id, fighter_1_id, fighter_2_id, winner_id, weight_class, win_method).
    - fight_rounds: Round-by-round quantitative metrics (round_id, fight_id, fighter_id, round_num, sig_strikes_landed, sig_strikes_att, takedowns_landed, takedowns_att, ctrl_time_sec).
- [ ] **Set Up Python Environment:** Establish a virtual environment (venv or conda) with key packages: pandas, sqlalchemy, scikit-learn, xgboost, and psycopg2 (or sqlite3).

Phase 2: Data Ingestion & ETL Pipeline

- **Goal:** Populate the SQL schema with clean, historical UFC fight statistics spanning multiple years.
- **Duration:** 2 – 3 Days

Tasks & Action Items

- [ ] **Acquire Data Source:** Download structured CSV datasets from Kaggle (or build a web scraper targeting structured sites like UFCStats.com using BeautifulSoup / Scrapy).
- [ ] **Data Cleansing & Normalization:**
    - Parse string ranges (e.g., convert "15 of 30" into separate integer columns: landed = 15, attempted = 30).
    - Convert fight time durations ("4:15") into total integer seconds (255).
    - Standardize fighter names and map unique UUIDs/IDs across all tables to avoid duplicate entries.
- [ ] **Database Ingestion:** Write an ETL Python script using pandas and sqlalchemy to push cleaned data into your SQL database (df.to_sql()).

Phase 3: Temporal Feature Engineering (Avoiding Data Leakage)

- **Goal:** Write SQL window functions and views that calculate pre-fight rolling historical stats strictly prior to each event date.
- **Duration:** 3 – 4 Days

Tasks & Action Items

- [ ] **Construct Pre-Fight Rolling Stats View:**
  - Calculate career averages up to fight date $T - 1$ using SQL window functions (AVG() OVER (PARTITION BY fighter_id ORDER BY event_date ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)).
  - Compute rolling performance over recent form (e.g., stats over last 3 fights).
- [ ] **Engineer Differential Features:**
  - **Physical Differentials:** reach_diff (Fighter A reach - Fighter B reach), height_diff, age_diff (calculated dynamically using event date and dob).
  - **Striking Metrics:** sig_strike_landed_per_min_diff, striking_accuracy_diff, striking_defense_diff.
  - **Grappling Metrics:** takedown_avg_per_15m_diff, takedown_accuracy_diff, takedown_defense_diff, control_time_ratio_diff.
  - **Inactivity & Momentum:** days_since_last_fight_diff, current_win_streak_diff.
- [ ] **Create Final ML Training View (fight_ml_features):** Combine all engineered differential features alongside a binary target label (fighter_1_wins: 1 if Fighter 1 won, 0 if Fighter 2 won).

Phase 4: Machine Learning Pipeline & Model Training

- **Goal:** Extract SQL feature matrices, train classification algorithms, and optimize for probabilistic win outputs.
- **Duration:** 2 – 3 Days

Tasks & Action Items

- [ ] **Data Extraction:** Connect Python to SQL database and pull the pre-computed feature matrix (pd.read_sql()).
- [ ] **Chronological Train/Test Split:** Split the dataset chronologically (e.g., train on pre-2023 fights, evaluate on 2023–2026 fights) to prevent future data leaking into past evaluations.
- [ ] **Baseline Model Implementation:** Train a baseline **Logistic Regression** model to establish a benchmark accuracy score.
- [ ] **Advanced Gradient Boosting:** Train **XGBoost** and **LightGBM** classifiers on the differential feature set.
- [ ] **Hyperparameter Optimization:** Perform grid/random search tuning on tree depth, learning rate, and regularization terms (max_depth, learning_rate, subsample).

Phase 5: Evaluation, Pattern Visualization & Inference

- **Goal:** Evaluate model performance against baseline benchmarks and build a workflow for predicting upcoming fights.
- **Duration:** 1 – 2 Days

Tasks & Action Items

- [ ] **Model Evaluation Metrics:** Assess the model using **Log Loss** (evaluating probability calibration) and **Brier Score** alongside standard accuracy.
- [ ] **Feature Importance Analysis:** Use SHAP (SHapley Additive exPlanations) values to identify which features drive predictions most (e.g., age gap vs. striking efficiency).
- [ ] **Upcoming Fight Prediction Pipeline:**
    - Input a new matchup (Fighter A vs. Fighter B).
    - Auto-query the database for both fighters' historical statistics.
    - Construct the differential vector and output predicted win probabilities (e.g., Fighter A: 61.4%, Fighter B: 38.6%).

🛠️ Recommended Tech Stack Summary

- **Database:** PostgreSQL or SQLite
- **ETL & Data Processing:** Python 3, Pandas, SQLAlchemy
- **Machine Learning:** Scikit-Learn, XGBoost, SHAP
- **Development Environment:** VS Code, Jupyter Notebooks